

数据库系统

Chapter 12

数据库架构

Database Architecture

详细学习笔记

考点总结 • 架构对比 • 案例分析

2026 年 6 月 21 日

基于课堂 PPT 课件整理

共 47 页 PPT • 全面覆盖考点

目录

1 数据库架构概述	2
1.1 课程定位	2
2 集中式系统与 C/S 架构	2
2.1 集中式数据库系统 (Centralized Database System)	2
2.2 客户端/服务器架构 (Client/Server, C/S)	2
2.3 C/S 架构中的服务器类型	3
3 并行数据库 (Parallel Databases)	3
3.1 并行数据库概念	3
3.2 四种并行架构 [p.p.10-13]	4
3.3 并行查询处理 [p.p.14-17]	5
3.4 并行性能度量 [p.p.18-19]	5
3.5 并行系统中的可用性与弹性 [p.p.20]	6
4 分布式数据库 (Distributed Databases)	6
4.1 分布式数据库系统概述 [p.p.21]	6
4.2 分布式数据库的优点与挑战 [p.p.22]	6
5 数据分片 (Sharding/Fragmentation)	7
5.1 分片类型	7
5.2 分片策略实践 [p.p.25]	7
5.3 数据复制 (Replication) [p.p.24]	8
6 分布式事务与 2PC 协议	8
6.1 分布式事务的挑战	8
6.2 两阶段提交协议 (Two-Phase Commit, 2PC)	8
6.3 三阶段提交 (Three-Phase Commit, 3PC)	9
7 CAP 定理	9
7.1 CAP 定理定义 [p.p.27]	9
7.2 CAP 的三类系统选择	10
8 BASE 与最终一致性	11
8.1 从 ACID 到 BASE [p.p.29]	11
8.2 BASE 模型三要素	11

8.3 最终一致性的变体 [p.p.30]	11
9 NoSQL 数据库	12
9.1 NoSQL 概述 [p.p.31]	12
9.2 四种 NoSQL 数据库类型 [p.p.32-35]	12
10 NewSQL 数据库 [p.p.36]	13
10.1 NewSQL 概念	13
10.2 NewSQL 架构特点	14
10.3 代表性 NewSQL 系统	14
11 云数据库 (Cloud Database)	15
11.1 DBaaS 概念 [p.p.38]	15
11.2 云数据库的部署模式	15
11.3 云数据库关键技术 [p.p.40]	15
12 案例研究：阿里巴巴数据库架构演进	16
12.1 阿里数据库体系的四个时代 [p.p.27]	16
12.2 第一阶段：淘宝初创 (2003–2004) [p.p.28-30]	16
12.3 第二阶段：辉煌时代——IOE 架构 (2005–2010) [p.p.31-32]	17
12.4 第三阶段：无冕之王——AliSQL (2011–2015) [p.p.33-34]	17
12.5 12 年历程回顾 [p.p.35]	18
12.6 第四阶段：新挑战新机遇——单元化 (2016 至今) [p.p.36-41]	18
12.7 第五阶段：OceanBase——走向自研 [p.p.43-47]	19
13 考点总览与复习建议	21
13.1 核心考点梳理	21
13.2 高频题型预测	21
13.3 复习建议	22

数据库架构概述

课程定位

数据库架构(Database Architecture)是数据库系统的顶层设计,决定了系统的部署方式、扩展能力和性能特征。随着数据量和并发量的增长,数据库架构从集中式系统逐步演化为并行、分布式、云原生等多种形态。

架构选型的核心考量因素:

- 性能需求: 吞吐量(Throughput)、延迟(Latency)
- 扩展性: 垂直扩展(Scale-Up) vs 水平扩展(Scale-Out)
- 可用性: 故障恢复时间(RTO)、数据丢失容忍度(RPO)
- 一致性: 强一致 vs 最终一致
- 成本: 硬件、运维、开发复杂度

集中式系统与 C/S 架构

集中式数据库系统 (Centralized Database System)

集中式系统将数据库完全运行在单一计算机上,是最基础的架构形式。

- 单机架构: 一台服务器运行 DBMS,所有数据和计算集中于此 [p.p.2]
- 终端—主机模式: 哑终端(Dumb Terminal)通过网络连接到主机,终端只负责显示 [p.p.3]
- 优点: 结构简单、管理方便、一致性有保障
- 缺点: 单点故障、扩展困难 (受限于单机硬件)、性价比低

客户端/服务器架构 (Client/Server, C/S)

C/S 架构将系统分为客户端和服务端两部分,实现功能分离 [p.p.4]。

- 两层 C/S 架构 [p.p.5]:
 - 客户端: 用户界面、业务逻辑 (胖客户端)
 - 服务器: 数据库管理、数据存储
 - 通信: SQL 请求通过网络传输

- 三层/多层架构 [p.p.6]:
 - 表示层(Presentation Tier): 用户界面
 - 应用层/中间件层(Application/Middle Tier): 业务逻辑
 - 数据层(Data Tier): 数据库服务器

三层架构的核心优势:

- 瘦客户端: 客户端只需浏览器或轻量应用
- 业务逻辑集中: 中间层统一管理, 易于维护升级
- 更好的扩展性: 各层可独立扩展
- 更高的安全性: 客户端不直接访问数据库

C/S 架构中的服务器类型

- 事务服务器(Transaction Server) [p.p.7]:
 - 也称为查询服务器(Query Server)
 - 客户端发送 SQL/事务请求, 服务器执行并返回结果
 - 典型系统: 主流 RDBMS(Oracle, MySQL, SQL Server)
- 数据服务器(Data Server) [p.p.8]:
 - 客户端请求数据页/文件, 在本地执行处理
 - 网络传输量更大, 但可分担服务器计算压力
 - 示例: 早期的文件服务器、某些 NoSQL 系统

并行数据库 (Parallel Databases)

并行数据库概念

并行数据库系统在多个处理器和磁盘间并行执行数据库操作, 以提升性能和吞吐量 [p.p.9]。

并行 vs 分布式的区别:

- 并行数据库: 多个处理器/磁盘在同一地点, 紧密耦合, 目标为加速单个查询
- 分布式数据库: 物理上分散在不同地点, 松耦合, 目标为数据共享与可用性

四种并行架构 [p.p.10-13]

1. 共享内存 (Shared Memory) [p.p.10]

- 所有处理器共享同一内存和磁盘
- 处理器间通过内存总线通信
- 优点：实现简单，负载均衡易
- 缺点：内存总线成为瓶颈，扩展性受限于总线带宽
- 典型：SMP(Symmetric Multi-Processing)服务器

2. 共享磁盘 (Shared Disk) [p.p.11]

- 所有处理器有独立内存，但共享磁盘系统(SAN/NAS)
- 处理器间通过网络或磁盘通信
- 优点：内存总线不再瓶颈，容错性好（单节点故障不影响其他）
- 缺点：磁盘访问竞争，扩展时磁盘成为瓶颈
- 典型：Oracle RAC(Real Application Clusters)

3. 无共享 (Shared Nothing) [p.p.12]

- 每个节点有独立的内存和磁盘，节点间通过网络通信
- 优点：扩展性最佳，无共享资源竞争
- 缺点：数据分区和负载均衡复杂，通信开销大
- 典型：Teradata、Greenplum、HBase

4. 层次架构 (Hierarchical) [p.p.13]

- 组合上述架构：顶层无共享，每个节点内部共享内存
- 优点：兼顾局部共享的高效和全局无共享的扩展性
- 典型：现代大规模集群常用此架构

四种架构对比表：

	共享内存	共享磁盘	无共享	层次
内存	全部共享	各自独立	各自独立	局部共享
磁盘	全部共享	全部共享	各自独立	局部共享
扩展性	差	中等	最好	很好
复杂性	低	中等	高	最高
通信方式	总线	网络/磁盘	网络	混合

并行查询处理 [p.p.14-17]

- 操作间并行(Inter-Operation Parallelism) [p.p.14]:
 - 查询执行计划的不同操作同时在不同处理器上执行
 - 流水线并行(Pipelined Parallelism): 下游操作消费上游操作的输出, 形成流水线
 - 独立并行(Independent Parallelism): 可独立执行的操作同时进行
- 操作内并行(Intra-Operation Parallelism) [p.p.15]:
 - 单个操作 (如排序、连接) 分解为多个子任务, 并行执行
 - 并行排序: 范围分区后各节点独立排序再合并
 - 并行连接(Parallel Join):
 - * 分区连接(Partitioned Join): 两表按相同分区函数分区, 各自执行连接
 - * 广播连接(Broadcast Join): 小表广播到所有节点, 各节点执行本地连接
- 数据分区策略 [p.p.16]:
 - 轮转分区(Round-Robin): 轮流分配记录, 分布均匀
 - 哈希分区(Hash Partitioning): 按哈希值分区, 支持等值查询高效定位
 - 范围分区(Range Partitioning): 按值范围分区, 支持范围查询

并行性能度量 [p.p.18-19]

并行系统的性能通过加速比和扩展比来衡量。

加速比 (Speedup): $\text{Speedup} = \frac{\text{小系统上执行时间}}{\text{大系统上执行时间}} = \frac{T_{\text{small}}}{T_{\text{large}}}$

线性加速比(Linear Speedup): N 个处理器时加速比 $= N$

亚线性加速比(Sub-linear Speedup): 加速比 $< N$, 因存在启动开销、偏斜、竞争

扩展比 (Scaleup): $\text{Scaleup} = \frac{\text{小系统上小任务执行时间}}{\text{大系统上大任务执行时间}} = \frac{T_{\text{small, small}}}{T_{\text{large, large}}}$
线性扩展比 (Linear Scaleup): 资源增加 N 倍同时任务量增加 N 倍, 执行时间不变

影响并行性能的关键因素 (考试重点):

- 启动开销 (Startup Cost): 启动并行任务的时间开销
- 干扰 (Interference): 共享资源的竞争
- 数据偏斜 (Skew): 数据分布不均匀导致部分节点负载过高
- 通信开销: 节点间数据传输的网络开销

并行系统中的可用性与弹性 [p.p.20]

当并行系统中某些节点失效时, 如何保证系统继续提供服务:

- 数据冗余: 复制数据到多个节点
- 在线恢复: 节点失效后自动重建数据
- 弹性伸缩 (Elasticity): 动态增减节点适应负载变化

分布式数据库 (Distributed Databases)

分布式数据库系统概述 [p.p.21]

分布式数据库是将数据分散存储在通过网络连接的多台计算机上的数据库系统。

分布式数据库的核心特征:

- 数据分布: 数据物理上分布在多个节点
- 逻辑统一: 对用户呈现为单一逻辑数据库
- 节点自治: 每个节点有独立的 DBMS
- 网络互联: 节点间通过消息传递协作

分布式数据库的优点与挑战 [p.p.22]

优点:

- 数据共享: 不同地点的用户可以访问共享数据
- 本地自治: 各节点可独立管理本地数据
- 高可用性: 单节点故障不影响全系统
- 可扩展性: 水平扩展, 添加节点提升容量

挑战：

- 网络开销：节点间通信增加延迟
- 数据一致性：维护多副本间的一致性困难
- 分布式查询优化：需要全局优化策略
- 事务管理：分布式事务的 ACID 保证复杂

数据分片 (Sharding/Fragmentation)

分片类型

数据分片是将一个大表的数据分散存储的技术。

- 水平分片(Horizontal Fragmentation) [p.p.23]:
 - 按行拆分，不同行的记录分布到不同节点
 - 每个分片包含相同的列，不同的行
 - `SELECT * FROM table WHERE shard_key = value`
- 垂直分片(Vertical Fragmentation) [p.p.23]:
 - 按列拆分，不同列分布到不同节点
 - 通过主键关联各分片记录
 - 适合将不常用的列分离到独立存储
- 混合分片(Hybrid Fragmentation) [p.p.23]:
 - 先垂直分片再水平分片，或先水平后垂直

分片策略实践 [p.p.25]

水平拆分的两种分类方法（来自 PPT 原文）：

1) 按时间分类：如果数据的时效性很强，可以认为所有数据中，20% 近期更新的数据能够满足业务 80% 的需求（“二八”定律）。例如，如果有 5 年的历史数据，就可以认为其中在 1 年内（20%）更新过的数据，能满足 80% 的业务需求。可以把表拆成两个表，分别存储 20% 和 80% 的数据。如果两张表仍然没有有效提高性能，还可以利用“二八”定律再次分割。

2) 按索引分类：当数据的时效性不明显时，可以按索引分类数据。所谓索引可以是任何可以用于分类的字段，比如部门编号、员工编号、工艺编号等等。例如表中存储了所有零件的

信息，但是在 80% 的情况下，1 号生产车间只会存取自己部门用到的零件。于是可以按照部门编号，把表分成多个。

数据复制 (Replication) [p.p.24]

- 主从复制(Master-Slave):
 - 写入只发生在主节点，从节点只读
 - 主节点将更新异步或同步复制到从节点
 - 优点：读写分离，读扩展性好
 - 缺点：主节点单点写入瓶颈，从节点可能有复制延迟
- 多主复制(Multi-Master):
 - 多个节点都可以写入
 - 需要冲突检测与解决机制
 - 优点：写入可扩展，无单点瓶颈
 - 缺点：冲突处理复杂，一致性维护困难
- 对等复制(Peer-to-Peer):
 - 无中心节点，所有节点地位平等
 - 基于 Gossip 协议等实现数据传播
 - 典型系统：Cassandra、Dynamo

分布式事务与 2PC 协议

分布式事务的挑战

分布式事务涉及跨多个节点的数据操作，需要保证全局 ACID 特性。核心问题是：如何在网络不可靠、节点可能故障的情况下，保证所有节点要么全部提交、要么全部回滚？

两阶段提交协议 (Two-Phase Commit, 2PC)

2PC 是保证分布式事务原子性的经典协议 [p.p.26]。

- 角色：
 - 协调者(Coordinator)：负责协调整个事务的提交

- 参与者(Participant/Cohort): 执行实际数据操作的节点
- 阶段一: 准备阶段(Prepare Phase):
 - 协调者向所有参与者发送 PREPARE 消息
 - 参与者执行事务操作, 写入日志但不提交
 - 参与者回复 READY (可以提交) 或 ABORT (需要回滚)
- 阶段二: 提交阶段(Commit Phase):
 - 如果所有参与者回复 READY: 协调者发送 COMMIT, 参与者提交
 - 如果任一参与者回复 ABORT 或超时: 协调者发送 ROLLBACK, 参与者回滚

2PC 的主要问题 (核心考点):

- 阻塞问题(Blocking): 协调者故障时, 参与者不确定应该提交还是回滚, 必须等待协调者恢复
- 协调者单点故障: 协调者故障导致整个协议停滞
- 性能开销: 两次网络往返 + 日志刷盘, 延迟高
- 乐观假设: 假设节点故障是暂时的, 不适合长时间网络分区

三阶段提交 (Three-Phase Commit, 3PC)

3PC 在 2PC 基础上增加了一个预提交阶段以减少阻塞 [p.p.26]。

- 阶段一: CAN_COMMIT? (询问是否可以提交)
- 阶段二: PRECOMMIT (预提交, 各节点已知最终要提交)
- 阶段三: DO_COMMIT (最终提交)

3PC 通过超时机制和状态传递, 确保即使协调者故障, 参与者也能通过选举新协调者继续完成协议, 但仍无法处理网络分区场景。

CAP 定理

CAP 定理定义 [p.p.27]

CAP 定理(CAP Theorem)由 Eric Brewer 于 2000 年提出, 是分布式系统设计中最重要的理论约束。

CAP 定理核心内容：一个分布式系统最多只能同时满足以下三个属性中的两个：

1. **一致性(Consistency)：**所有节点在同一时刻看到相同的数据（读操作总返回最新写入的值）
2. **可用性(Availability)：**每个请求都能获得（非错误的）响应，但不保证响应包含最新数据
3. **分区容错(Partition Tolerance)：**系统在网络分区(Network Partition)（节点间消息丢失或延迟）的情况下仍能继续运行

CAP 的三类系统选择

由于网络分区在分布式系统中不可避免，实际选择通常在 CP 和 AP 之间：

- **CA 系统（放弃 P）：**
 - 仅在无网络分区的情况下存在
 - 典型：传统单机 RDBMS、共享磁盘集群
 - 现实中局域网也会有分区，严格 CA 几乎不存在
- **CP 系统（放弃 A） [p.p.28]：**
 - 发生网络分区时，牺牲可用性以保证一致性
 - 系统可能拒绝部分请求以保证数据不出现不一致
 - 典型：HBase、MongoDB（强一致配置）、ZooKeeper
- **AP 系统（放弃 C） [p.p.28]：**
 - 发生网络分区时，牺牲强一致性以保证可用性
 - 系统继续接受请求，允许暂时的不一致
 - 典型：Cassandra、DynamoDB、CouchDB

CAP 定理的关键理解（易错点）：

- ”三选二”是对分区发生时的描述，不是任何时候都必须牺牲某一个
- 分区容错不是”可选”的——在广域网中网络分区不可避免，实际上必须在 C 和 A 之间权衡
- 一致性和可用性是一个连续谱，不是二分选择
- 很多系统提供可调节的一致性级别

BASE 与最终一致性

从 ACID 到 BASE [p.p.29]

当放弃强一致性（C）以换取更好的可用性和性能时，引入 BASE 模型。

ACID vs BASE 对比：

属性	ACID(传统 RDBMS)	BASE (NoSQL/分布式)
一致性模型	强一致性	最终一致性
可用性	可能降级	基本可用
事务支持	严格事务	柔性事务
适用场景	金融、交易	社交、日志、推荐
分区策略	优先一致性	优先可用性

BASE 模型三要素

- 基本可用(Basically Available):
 - 系统大部分时间可用
 - 故障时允许部分功能降级而非完全不可用
 - 例如：响应可能变慢，某些非核心功能暂时不可用
- 软状态(Soft State):
 - 系统状态可以随时间变化，即使没有新的输入
 - 数据副本间存在不一致的中间状态
 - 不需要即时一致
- 最终一致性(Eventual Consistency):
 - 如果系统不再接收新的更新，经过足够时间后，所有副本将收敛到一致状态
 - 不保证读操作立即反映最新写入
 - 典型的最终一致性系统：DNS、Cassandra

最终一致性的变体 [p.p.30]

- 因果一致性(Causal Consistency): 有因果关系的写入按顺序被看到

- 读己之写(Read-Your-Writes): 客户端总能看到自己写入的最新数据
- 会话一致性(Session Consistency): 在同一个会话内保证读己之写
- 单调读(Monotonic Read): 不会读到比之前更旧的数据
- 单调写(Monotonic Write): 同一客户端的写入按顺序执行

NoSQL 数据库

NoSQL 概述 [p.p.31]

NoSQL(Not Only SQL)是一类非关系型数据库的统称, 放弃了传统关系模型和 SQL 接口以获得更好的扩展性、灵活性和性能。

NoSQL 兴起的原因:

- 大数据时代的数据量爆炸性增长
- 半结构化和非结构化数据的处理需求
- 互联网应用对高并发、低延迟的需求
- 水平扩展比垂直扩展更经济
- 敏捷开发对灵活数据模型的需求

四种 NoSQL 数据库类型 [p.p.32-35]

1. 键值存储(Key-Value Store) [p.p.32]

- 最简单的数据模型: $\text{Key} \rightarrow \text{Value}$
- Value 可以是任意类型 (字符串、JSON、二进制)
- 接口简单: `Get(key)`、`Put(key, value)`、`Delete(key)`
- 典型系统: Redis、Amazon DynamoDB、Riak
- 适用: 缓存、会话管理、简单查找

2. 文档数据库(Document Store) [p.p.33]

- 存储半结构化的文档(JSON/BSON/XML)
- 支持对文档内部字段的查询和索引
- 文档可以有不同结构, 灵活扩展
- 典型系统: MongoDB、CouchDB、Couchbase
- 适用: 内容管理、用户档案、产品目录

3. 列族存储(Column-Family Store) [p.p.34]

- 数据按列族组织，而非按行
- 稀疏列：每行可以有不同列，空列不占存储
- 按列存储便于聚合和扫描
- 典型系统：HBase、Cassandra、Bigtable
- 适用：日志分析、时序数据、大规模扫描

4. 图数据库(Graph Database) [p.p.35]

- 数据以节点(Node)和边(Edge)形式存储
- 节点和边都可以有属性
- 擅长处理复杂关联关系查询
- 典型系统：Neo4j、JanusGraph、Amazon Neptune
- 适用：社交网络、推荐系统、知识图谱、反欺诈

NoSQL 四种类型对比：

特征	键值	文档	列族	图
数据模型	Key-Value	JSON 文档	列族	节点 + 边
查询能力	仅 Key 查询	字段级查询	列级扫描	图遍历
一致性	最终/可调	最终/强	最终/可调	强一致
扩展性	最好	很好	最好	中等
典型场景	缓存/会话	内容管理	时序/日志	社交/推荐

NewSQL 数据库 [p.p.36]

NewSQL 概念

NewSQL 是一类新型关系数据库系统，旨在同时提供 NoSQL 的扩展性和传统 SQL 数据库的 ACID 事务保证。

NewSQL = NoSQL 的扩展性 + SQL 的 ACID 保证

NewSQL 试图打破“NoSQL 才有扩展性，SQL 才有事务”的二分法，提供两者的最佳组合。

NewSQL 架构特点

- 无共享架构：基于 Shared-Nothing 的分布式架构
- SQL 接口：完整支持 SQL 标准和关系模型
- ACID 事务：严格的事务保证，支持分布式事务
- 多版本并发控制(MVCC)：高并发无锁读
- 自动分片：数据自动分区和再平衡
- 共识协议：基于 Paxos/Raft 保证一致性

代表性 NewSQL 系统

- Google Spanner [p.p.37]:
 - 全球分布式的关系数据库
 - 使用 TrueTime API 提供外部一致性
 - 基于 Paxos 的多副本同步复制
- CockroachDB [p.p.37]:
 - 开源、云原生的分布式 SQL 数据库
 - 基于 Raft 的一致性复制
 - 兼容 PostgreSQL 协议
- TiDB [p.p.37]:
 - 开源分布式 HTAP 数据库（中国开发）
 - 兼容 MySQL 协议
 - TiKV 存储层 + TiDB 计算层分离
- OceanBase [p.p.37]（详见后文阿里案例）:
 - 蚂蚁集团自研分布式数据库
 - 基于 Paxos 的高可用方案
 - 支持支付宝核心交易场景

云数据库 (Cloud Database)

DBaaS 概念 [p.p.38]

数据库即服务(Database as a Service, DBaaS)将数据库部署在云平台上, 用户无需管理底层基础设施。

DBaaS 的核心价值:

- 免运维: 云服务商负责安装、配置、备份、扩缩容
- 弹性伸缩: 按需分配资源, 自动扩展
- 高可用内置: 多 AZ(Availability Zone)部署、自动故障切换
- 按量付费: 从 CAPEX(资本支出)转为 OPEX(运营支出)
- 安全合规: 加密、审计、合规认证

云数据库的部署模式

- 虚拟机部署 [p.p.39]:
 - 在云 VM 上自建数据库
 - 灵活性最高, 但运维负担重
- 托管服务(Managed Service) [p.p.39]:
 - 云厂商管理数据库实例
 - 典型: AWS RDS、阿里云 RDS、Azure SQL Database
- Serverless 数据库 [p.p.39]:
 - 按实际使用计费, 自动扩缩, 闲置时可缩至零
 - 典型: AWS Aurora Serverless、Azure SQL Serverless
- 云原生数据库 [p.p.40]:
 - 为云架构从零设计, 存算分离
 - 典型: AWS Aurora、Google Cloud Spanner、阿里云 PolarDB

云数据库关键技术 [p.p.40]

- 存算分离: 计算层和存储层独立扩展
- 多租户隔离: 资源共享与安全隔离的平衡

- 自动故障转移：跨 AZ/Region 的高可用
- 读写分离：主实例处理写，只读副本分担查询
- 全球部署：跨区域数据分布与一致性的权衡

案例研究：阿里巴巴数据库架构演进

本章 PPT 后半部分（p.25–p.47）以阿里巴巴为案例，详细展示了数据库架构从单机到分布式、从商业产品到自研系统的完整演进历程。这是理解数据库架构实践的最佳案例。

阿里数据库体系的四个时代 [p.p.27]

时代	时间	核心架构	关键词
淘宝初创	2003–2004	MySQL 单机 → Oracle	单机房、单机
辉煌时代	2005–2010	IOE 架构	IBM 小型机、Oracle、EMC 存储
无冕之王	2011–2015	去 IOE → AliSQL	垂直拆分、水平拆分、异地双活
新挑战新机遇	2016–至今	单元化 → Ocean-Base	异地多活、云化、自研

第一阶段：淘宝初创（2003–2004）[p.p.28-30]

- 初期架构 [p.p.29]:
 - Apache + mod_php4 + Pear DB
 - MySQL 主从复制架构(Master-Slave)
 - 架构简单：一个 Master 负责读写，多个 Slave 只读
- 遇到的问题 [p.p.30]:
 - 单机 MySQL 数据库迅速达到瓶颈
- 解决方案 [p.p.30]:
 - MySQL 迁移到 Oracle，并逐步升级硬件
 - 升级到小型机、高端存储
 - 最终形成 IOE 架构：IBM（小型机）+ Oracle（数据库）+ EMC（存储）

- 效果 [p.p.30]:
 - 支撑了淘宝 2004 到 2009 年发展高峰

第二阶段：辉煌时代——IOE 架构 (2005–2010) [p.p.31-32]

- IOE 架构特点 [p.p.31]:
 - I(IBM): 高价小型机作为数据库服务器
 - O(Oracle): 集中式数据库，存放所有核心业务数据
 - E(EMC): 高端存储设备，通过 FC 光纤连接
 - 双机房部署，Redo Log 远程复制实现数据零丢失
- IOE 架构的问题 [p.p.32]:
 - 扩展性问题：垂直扩展走到了极限，无法水平扩展
 - 可用性：集中式架构，稳定性面临挑战
 - 掌控力缺失：闭源的 Oracle、封闭的小型机和存储设备
 - 成本高昂：IBM 小型机和 EMC 存储价格昂贵

”去 IOE”运动的核心含义 [p.p.26]:

- 去 I：废弃以 IBM 为代表的高价小型机
- 去 O：废弃以 Oracle 为代表的集中式数据库
- 去 E：废弃以 EMC 为代表的高端存储设备
- 目标：用 x86 服务器 + 开源软件 + 分布式架构替代昂贵的商业一体机方案

第三阶段：无冕之王——AliSQL (2011–2015) [p.p.33-34]

- 从 IOE 走向 AliSQL 分布式架构 [p.p.33]:
 - 第一次推动中国数据库产业变革
 - 获得无限掌控力——从闭源商业软件转向开源自研
 - 数据库限流：第一次自己的命运自己掌握
 - 热点更新优化：定制优化热点商品减库存业务场景
 - 线程池特性优化：定制优化高连接数并发场景
- AliSQL 架构规模 [p.p.34]:
 - 上百组 MySQL 集群

- 大规模水平分片
- 支持淘宝核心交易链路

12 年历程回顾 [p.p.35]

年份	关键事件	技术意义
2003	淘宝网创建	单机 MySQL 起步
2004	开始尝试 MySQL	验证 MySQL 可行性
2005	引入 Oracle	对业务增长的积极响应
2006-07	引入小型机	垂直扩展路径
2008-09	硬件不断升级	集中式架构的极限逼近
2010	垂直拆分完成	按业务模块拆分数数据库
2011	水平拆分完成	按数据维度进一步拆分
2012	商品库完成去 IOE	核心业务率先完成转型
2013	淘宝全网完成去 IOE	全站实现分布式架构
2014	支付宝交易完成 OB 改造	金融级分布式数据库上线

第四阶段：新挑战新机遇——单元化 (2016 至今) [p.p.36-41]

- 面临的新挑战 [p.p.36]:
 - 资源限制：一个城市已经不能满足需求
 - 容灾需求：单地域机房风险，需要异地多活
 - AliSQL 瓶颈：分表数量庞大，集群拆分接近极限
 - 业务开发复杂度：路由、关联、聚合、订正处理复杂
- 单元化架构 [p.p.37]:
 - 按用户分流(CDN 分发)到不同单元中心
 - 每个单元是完整的业务处理闭环：
 - * 接入层 → 服务层 → 缓存层 → 数据层
 - 单元内同步调用（低延迟），单元间异步消息
 - 数据同步通过 DRC(Data Replication Center)
 - 中心备份保证数据安全
- 对应用的挑战及解法 [p.p.38]:
 - 延迟差异：

- * 同一机房：0.2ms
- * 同一城市：1-5ms
- * 跨城市：5ms-100ms
- 单元内封闭：尽量在单元内完成请求，减少跨单元调用
- 延迟影响：几百次跨单元调用即可导致吞吐量显著下降
- 数据复制延迟：跨单元数据同步存在时间差
- 对数据库的挑战及解法 [p.p.39]:
 - 数据买家维度拆分：按买家 ID 进行数据分区
 - 数据质量保障：确保分片后数据完整性
 - 单元间 DRC 数据复制：实时数据同步
 - 数据多点写入风险：多单元同时更新数据的一致性
 - 数据复制一致性保障：正确性验证和修复机制
- 单元化效益 [p.p.40]:
 - 稳定性提升：故障影响范围缩小到单个单元
 - 变更范围可控：变更先在单个单元验证
 - 故障恢复时间缩短：快速隔离和恢复
 - 伸缩能力增强：摆脱机房资源限制
 - 伸缩规模再次增强：可扩展到多地域
- 单元化项目时间线 [p.p.41]:
 - 2013-05：项目启动
 - 2013-08：同城两单元上线
 - 2014-08：异地双活实现
 - 2014-11：经过双 11 洗礼验证
 - 2015-08：多地域更远距离部署完成
 - 横跨三年的宏大项目

第五阶段：OceanBase——走向自研 [p.p.43-47]

- OceanBase 核心架构 [p.p.43]:
 - 数据存储：多机磁盘存储基线数据(Baseline Data)

- **修改增量：**单机内存存储增量更新(Incremental Updates)
- **读操作：**合并基线数据 + 增量数据返回结果
- **写操作：**先写入内存增量，定期合并到基线
- **存储介质：**内存(RAM) + 固态硬盘(SSD)

• **OceanBase 组件架构 [p.p.44]:**

- **RootServer：**集群管理、元数据存储
- **UpdateServer：**处理写请求，管理修改增量
- **ChunkServer：**存储基线数据分片
- **MergeServer：**接收 SQL 请求，合并基线 + 增量数据返回

• **基于 Paxos 的高可用方案 [p.p.45]:**

- 以不可靠部件提供可靠服务
- 小于半数的分区容忍性
- 较高的可用性(最大 35s 不可用)
- 强一致性

组件	异常描述	影响
RootServer	宕机/程序异常退出	35s 不可用
UpdateServer	宕机/程序异常退出	约 25s 不可用
MergeServer	宕机/程序异常退出	1 分钟/少量读超时
ChunkServer	宕机/程序异常退出	1 分钟/少量读超时
主集群不可用	机房断网	35s 不可用

• **OceanBase 发展历程 [p.p.46]:**

- 2010.6: 项目启动
- 2011: V0.1 支持淘宝收藏夹
- 2012: V0.3 支持广告报表
- 2013: V0.4 支持 SQL
- 2014: V0.5 支持支付宝交易
- 2015–2016: V1.0 全新架构发布，支持淘宝交易
- 2017 至今: V1.x 全面覆盖核心业务

考点总览与复习建议

核心考点梳理

编号	考点	关键内容	页码
1	集中式 vs C/S	三层架构、事务服务器 vs 数据服务器	p.2-8
2	并行架构四种	共享内存/磁盘/无共享/层次	p.10-13
3	并行查询	操作间并行、操作内并行、分区策略	p.14-17
4	性能度量	Speedup、Scaleup 公式及影响因素	p.18-19
5	数据分片	水平/垂直/混合分片、二八定律实践	p.23-25
6	数据复制	主从/多主/对等	p.24
7	分布式事务	2PC 协议、阻塞问题、3PC	p.26
8	CAP 定理	三选二、CP vs AP 系统	p.27-28
9	BASE 模型	基本可用/软状态/最终一致性	p.29-30
10	NoSQL 四类型	键值/文档/列族/图、各自特点	p.31-35
11	NewSQL	Spanner/CockroachDB/TiDB/Cassandra	p.36-37
12	云数据库	DBaaS、存算分离、Serverless	p.38-40
13	阿里演进案例	IOE → 去 IOE → AliSQL → 单元化 → OceanBase	p.25-47

高频题型预测

1. 简答题:

- 比较四种并行数据库架构的优缺点
- 解释 CAP 定理，并举例说明 CP 和 AP 系统
- 描述 2PC 协议的执行过程及其缺陷
- 对比 NoSQL 四种类型的适用场景

2. 计算题:

- 给定加速比数据, 判断线性/亚线性/超线性加速比
- 计算 Scaleup 值并分析系统扩展性

3. 论述/案例分析:

- 结合阿里去 IOE 案例, 分析集中式架构向分布式架构转型的动因和挑战
- 设计一个电商系统的数据库架构, 考虑分片、复制、一致性策略

复习建议

复习要点:

1. 理解概念之间的关系: 集中式 $\rightarrow$ C/S $\rightarrow$ 并行 $\rightarrow$ 分布式 $\rightarrow$ 云, 这是一个从简单到复杂的演进过程
2. 掌握架构对比: 四种并行架构在处理器、内存、磁盘三个维度上的差异
3. 熟记公式: Speedup 和 Scaleup 的定义与计算
4. 理解 CAP 与 BASE: 这是分布式系统设计的理论基础, 必考
5. 区分 NoSQL 类型: 记住每种类型的代表系统和使用场景
6. 阅读阿里案例: 这是架构演进的经典现实案例, 有助于理解理论知识在实际中的应用
7. 动手画图: 四种并行架构、2PC 流程、单元化架构最好能手绘

祝考试顺利!

本笔记基于课堂 PPT 47 页精心整理

涵盖全部考点, 结构清晰, 便于复习